**UNIVERSITAS MULTI DATA PALEMBANG**
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

**TEMPLAT LAPORAN**

# TUGAS 2

## Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

| | |
|---|---|
| **Nama dan NIM** | Putri Reda Maulidia 2428240086 |
| **Kelas** | SI5B |
| **Judul project / topik API** | Topik 21 : Kebencanaan (Laporan Bencana) |
| **Tautan repositori** | https://github.com/putrirdm/tugas1-restful-2428240086 |
| **Dosen pengampu** | Nur Rachmat, M.Kom. |
| **Tanggal pengumpulan** | Kamis, 22 Oktober 2026 |

# Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

## A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

## B. Bukti yang diminta

| No | Bukti | Bagian | Jenis |
|---|---|---|---|
| 1 | Daftar request dalam Collection | Bab 2 | Screenshot (wajib) |
| 2 | Cuplikan test script | Bab 3 | Kode teks (wajib) |
| 3 | Ringkasan Collection Runner | Bab 4 | Screenshot (wajib) |
| 4 | Hasil Newman di terminal | Bab 5 | Screenshot (pengayaan) |

## C. Syarat tangkapan layar

| Bukti | Yang harus terlihat |
|---|---|
| Collection | Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar. |
| Ringkasan Runner | Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal). |
| Newman | Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir. |
| Larangan | Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit. |

> **Sebelum mengambil screenshot Runner**
> Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

# Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

## 1.1 Deskripsi singkat API

**Penjelasan:**

*Jelaskan dalam dua sampai tiga kalimat: topik API Anda, entitas utamanya, dan teknologi yang dipakai (misalnya Express.js dengan struktur Route-Controller-Model).*

API yang saya buat bertopik kebencanaan, yaitu API untuk mengelola data laporan bencana seperti banjir, longsor, kebakaran, dan gempa. Entitas utamanya adalah disaster-reports (Laporan Bencana) yang memiliki data jenis bencana, lokasi, tanggal, tingkat keparahan, dan jumlah pengungsi. API ini dibuat menggunakan Node.js dan Express.js dengan struktur Route, Controller, dan Model, serta dilengkapi middleware untuk API Key dan penanganan error.

## 1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

| No | Metode | Alamat | API key | Status sukses | Fungsi |
|---|---|---|---|---|---|
| 1 | GET | {{base_url}}/disaster-reports | | 200 | Menampilkan seluruh data laporan bencana yang tersimpan. |
| 2 | GET | {{base_url}}/disaster-reports/{{bencanaId}} | | 200 | Menampilkan detail laporan bencana berdasarkan ID tertentu. |
| 3 | POST | {{base_url}}/disaster-reports | IYA | 201 | Menambahkan laporan bencana baru ke dalam sistem. |
| 4 | PUT | {{base_url}}/disaster-reports/{{bencanaId}} | IYA | 200 | Memperbarui data laporan bencana berdasarkan ID tertentu. |
| 5 | DELETE | {{base_url}}/disaster-reports/{{bencanaId}} | IYA | 204 | Menghapus laporan bencana berdasarkan ID tertentu. |
| 6 | GET | {{base_url}}/disaster-reports/{{bencanaId}} | | 404 | Memastikan data laporan bencana yang telah dihapus tidak dapat ditemukan lagi. |
| 7 | POST | {{base_url}}/disaster-reports | | 401 | Menguji keamanan API dengan mengirim permintaan tanpa API Key dan memastikan sistem menolaknya. |

| No | Metode | Alamat | API key | Status sukses | Fungsi |
|---|---|---|---|---|---|
| 8 | POST | {{base_url}}/disaster-reports | IYA | 400 | Menguji validasi dengan mengirim data laporan bencana yang tidak lengkap atau field wajib yang belum diisi. |
| 9 | POST | {{base_url}}/disaster-reports | IYA | 400 | Menguji penanganan kesalahan dengan mengirim data berformat JSON yang tidak valid. |

## 1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

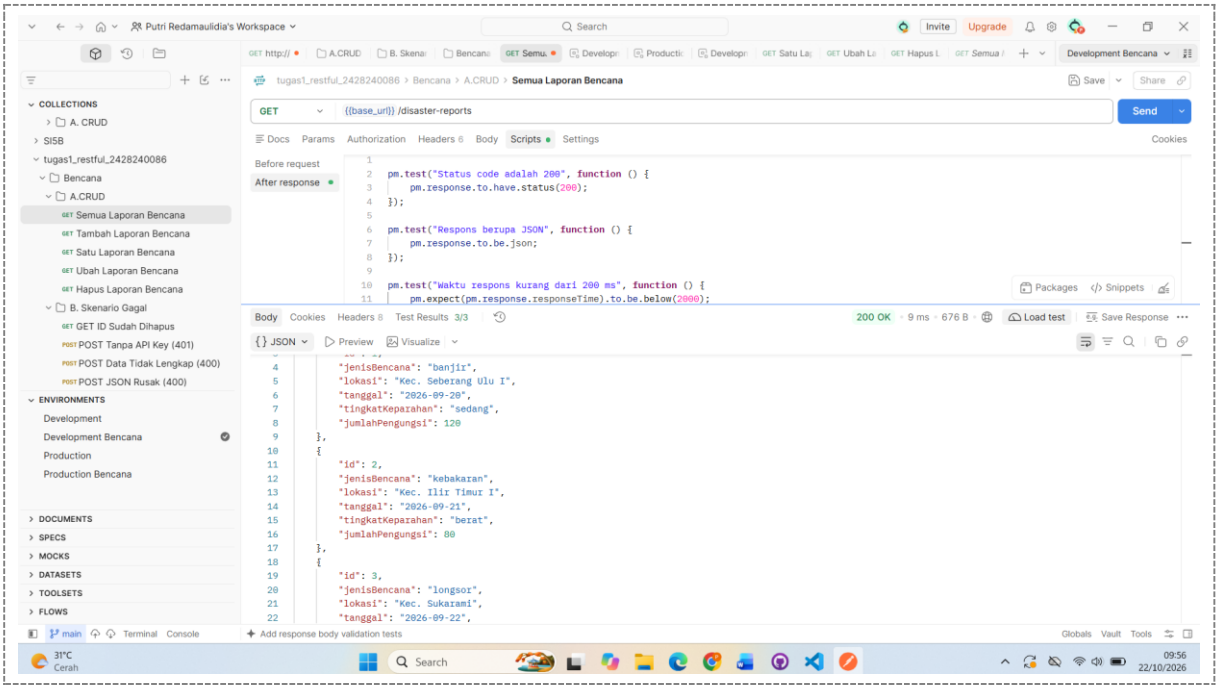
| No | Nama variabel | Nilai (contoh) | Fungsi |
|---|---|---|---|
| 1 | base_url | http://localhost:300 | Alamat server |
| 2 | api_key | tugas1_si5b_paw2 | Nilai header kunci API |
| 3 | bencanaId | (kosong, diisi script) | ID dari respons POST |
| 4 | | | |

# Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

## 2.1 Daftar request dalam Collection

**Harus terlihat pada screenshot:** nama Collection dan seluruh request di dalam folder. Buka semua folder.




*Gambar 1. Collection beserta seluruh request*

**Penjelasan:**

*Jelaskan cara Anda mengelompokkan request (misalnya folder CRUD dan folder skenario gagal) dan mengapa urutannya seperti itu.*

Saya mengelompokkan request di Postman menjadi dua folder, yaitu CRUD dan Skenario Gagal. Folder CRUD berisi request untuk melihat, menambah, mengubah, dan menghapus data laporan bencana. Sementara itu, folder Skenario Gagal berisi pengujian ketika API Key tidak ada, data yang dikirim tidak lengkap, format JSON salah, dan data tidak ditemukan. Saya menyusun urutannya seperti itu supaya lebih mudah saat melakukan pengujian, dimulai dari fungsi yang normal terlebih dahulu, baru dilanjutkan dengan pengujian jika terjadi kesalahan.

## 2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

| No | Nama request | Metode | Alamat (pakai {{base_url}}) | Folder |
|---|---|---|---|---|
| 1 | Semua Laporan Bencana | GET | {{base_url}}/disaster-reports | A. CRUD |
| 2 | Tambah Laporan Bencana | POST | {{base_url}}/disaster-reports | A. CRUD |
| 3 | Satu Laporan Bencana | GET | {{base_url}}/disaster-reports/{{bencanaId}} | A. CRUD |

| No | Nama request | Metode | Alamat (pakai {{base_url}}) | Folder |
|---|---|---|---|---|
| 4 | Ubah Laporan Bencana | PUT | {{base_url}}/disaster-reports/{{bencanaId}} | A. CRUD |
| 5 | Hapus Laporan Bencana | DELETE | {{base_url}}/disaster-reports/{{bencanaId}} | A. CRUD |
| 6 | GET ID Sudah Dihapus | GET | {{base_url}}/disaster-reports/{{bencanaId}} | B. Skenario Gagal |
| 7 | POST Tanpa API Key (401) | POST | {{base_url}}/disaster-reports | B. Skenario Gagal |
| 8 | POST Data Tidak Lengkap (400) | POST | {{base_url}}/disaster-reports | B. Skenario Gagal |
| 9 | POST JSON Rusak (400) | POST | {{base_url}}/disaster-reports | B. Skenario Gagal |
| 10 | | | | |

# Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

## 3.1 Rekap assertion per request

| No | Nama request | Assertion status code | Assertion struktur atau isi JSON |
|---|---|---|---|
| 1 | Semua Laporan Bencana | pm.response.to.have.status(200); | pm.**test**("Status code adalah 200", **function** () {<br>  pm.response.to.have.status(200);<br>});<br><br>pm.**test**("Respons berupa JSON", **function** () {<br>  pm.response.to.be.json;<br>});<br><br>pm.**test**("Waktu respons kurang dari 200 ms", **function** () {<br>  pm.expect(pm.response.responseTime).to.be.below(2000);<br>}); |
| 2 | Tambah Laporan Bencana | pm.response.to.have.status(201) | pm.**test**('Status code adalah 201', **function** () {<br> pm.response.to.have.status(201);<br>});<br><br>pm.**test**('Data baru memiliki id dan jenis bencana sesuai kiriman', **function** () {<br> const jsonData **=** pm.response.json();<br> pm.expect(jsonData).to.have.property('id');<br> pm.expect(jsonData.nama).to.eql('banjir');<br>});<br><br>**if** (pm.response.code **===** 201) {<br> pm.environment.**set**('bencanaId', pm.response.json().id);<br>} |

| No | Nama request | Assertion status code | Assertion struktur atau isi JSON |
|---|---|---|---|
| 3 | Satu Laporan Bencana | pm.response.to.have.status(200); | |
| 4 | Ubah Laporan Bencana | pm.response.to.have.status(200); | pm.**test**("Status code adalah 200 OK", **function** () {<br>  pm.response.to.have.status(200);<br>});<br><br>pm.**test**("Respons berupa JSON", **function** () {<br>  pm.response.to.be.json;<br>});<br><br>pm.**test**("Data laporan berhasil diubah", **function** () {<br>  const jsonData **=** pm.response.json();<br><br>  pm.expect(jsonData).to.have.property("id");<br>  pm.expect(String(jsonData.id)).to.eql(<br>  String(pm.environment.**get**("bencanaId"))<br>  );<br>});<br><br>pm.**test**("Tingkat keparahan sudah berubah", **function** () {<br>  const jsonData **=** pm.response.json();<br><br>  pm.expect(jsonData.tingkatKeparahan).to.eql("berat");<br>}); |
| 5 | Hapus Laporan Bencana | pm.response.to.have.status(204) | |
| 6 | GET ID Sudah Dihapus | pm.response.to.have.status(404) | |

| No | Nama request | Assertion status code | Assertion struktur atau isi JSON |
|---|---|---|---|
| 7 | POST Tanpa API Key (401) | pm.response.to.have.status(401) | pm.**test**("Status code adalah 401", **function** () {<br>  pm.response.to.have.status(401);<br>});<br><br>pm.**test**("Pesan error API Key sesuai", **function** () {<br>  const jsonData **=** pm.response.json();<br><br>  pm.expect(jsonData.message).to.eql(<br>  "API key tidak valid atau tidak diberikan"<br>  );<br>}); |
| 8 | POST Data Tidak Lengkap (400) | pm.response.to.have.status(400); | pm.**test**('Status code adalah 400', **function** () {<br> pm.response.to.have.status(400);<br>});<br><br>pm.**test**('Pesan error menyebut field wajib', **function** () {<br> pm.expect(pm.response.json().message).to.include('wajib diisi');<br>}); |
| 9 | POST JSON Rusak (400) | pm.response.to.have.status(400); | pm.**test**('Status code adalah 400', **function** () {<br> pm.response.to.have.status(400);<br>});<br><br>pm.**test**('Respons tetap berupa JSON, bukan HTML', **function** () {<br> pm.response.to.be.json;<br> pm.expect(pm.response.json().message).to.eql('Format JSON tidak valid');<br>}); |
| 10 | | | |

## 3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
// Test script request GET: [nama request]
// Tempel kode test script Anda (teks, bukan gambar), lalu hapus baris ini.
GET: Semua Laporan Bencana

pm.test("Status code adalah 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Respons berupa JSON", function () {
    pm.response.to.be.json;
});

pm.test("Waktu respons kurang dari 200 ms", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});
```

**Penjelasan:**

*Jelaskan fungsi setiap `pm.test`: apa yang diperiksa dan mengapa pemeriksaan itu penting.*

Script pertama digunakan untuk mengecek apakah request berhasil menampilkan semua laporan bencana dengan status 200. Script kedua memeriksa apakah data yang diterima berbentuk JSON, supaya respons dari API sesuai dengan format yang diharapkan. Script ketiga mengecek waktu respons agar tidak terlalu lama, sehingga API bisa menampilkan data dengan cepat.

## 3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
// Test script request POST: [nama request]
// Tempel kode test script Anda (teks, bukan gambar), lalu hapus baris ini.
POST: Tambah Laporan Bencana

pm.test('Status code adalah 201', function () {
  pm.response.to.have.status(201);
});

pm.test('Data baru memiliki id dan jenis bencana sesuai kiriman', function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData).to.have.property('id');
  pm.expect(jsonData.nama).to.eql('banjir');
});

if (pm.response.code === 201) {
  pm.environment.set('bencanaId', pm.response.json().id);
}
```

**Penjelasan:**

*Jelaskan bagaimana data dari respons disimpan, request mana yang memakainya, dan apa yang terjadi bila urutan request ditukar.*

Script tersebut digunakan untuk memeriksa apakah proses menambahkan laporan bencana berhasil dilakukan. Pada pm.test pertama, sistem memeriksa apakah status respons adalah 201 Created, yang berarti data baru berhasil ditambahkan. Pada pm.test kedua, sistem memeriksa apakah data yang dikirim memiliki id dan jenis bencana sesuai dengan yang diharapkan, supaya kita tahu data yang ditambahkan sudah benar. Setelah itu, jika respons berstatus 201, ID laporan yang baru dibuat disimpan ke variabel bencanaId agar bisa digunakan untuk request lain, seperti melihat, mengubah, atau menghapus laporan tersebut.

# Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

## 4.1 Ringkasan Collection Runner

**Harus terlihat pada screenshot:** nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.

**[ Tempel screenshot di sini ]**

*Gambar 2. Ringkasan hasil Collection Runner*

*Gambar 2. Ringkasan hasil Collection Runner*

**Penjelasan:**

*Jelaskan hasilnya: berapa request dan test yang dijalankan, apakah semuanya lulus, dan apa yang Anda lakukan sebelum menjalankan Runner (misalnya menyalakan ulang server).*

## 4.2 Rekap angka hasil

| | |
|---|---|
| **Nama Collection** | RESTful API |
| **Environment yang dipakai** | Development |
| **Jumlah request dijalankan** | 9 |
| **Jumlah test lulus** | 12 |
| **Jumlah test gagal** | |
| **Tanggal dan jam pengujian** | 22/10/2026, jam : 01:09:12 PM |

## 4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

| No | Request | Kegagalan (pesan) | Penyebab | Perbaikan |
|---|---|---|---|---|
| 1 | Semua Laporan Bencana | Tidak ada kegagalan | | |

| No | Request | Kegagalan (pesan) | Penyebab | Perbaikan |
|---|---|---|---|---|
| 2 | Tambah Laporan Bencana | Tidak ada kegagalan | | |
| 3 | Satu Laporan Bencana | Tidak ada kegagalan | | |
| 4 | Ubah Laporan Bencana | Tidak ada kegagalan | | |
| 5 | Hapus Laporan Bencana | Tidak ada kegagalan | | |
| 6 | GET ID Sudah Dihapus | Tidak ada kegagalan | | |
| 7 | POST Tanpa API Key (401) | Tidak ada kegagalan | | |
| 8 | POST Data Tidak Lengkap (400) | Tidak ada kegagalan | | |
| 9 | POST JSON Rusak (400) | Tidak ada kegagalan | | |

# Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

## 5.1 Hasil Newman

**Harus terlihat pada screenshot:** perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.

**[ Tempel screenshot di sini ]**

*Gambar 3. Hasil menjalankan Collection dengan Newman*

*Gambar 3. Hasil menjalankan Collection dengan Newman*

**Penjelasan:**

*Tulis perintah yang Anda jalankan dan jelaskan arti exit code (0 atau 1) pada hasil Anda.*

# Bab 6 Repositori dan Berkas

| | |
|---|---|
| **Tautan repositori** | https://github.com/putrirdm/tugas1-restful-2428240086 |
| **Lokasi berkas Collection (.json)** | |
| **Lokasi berkas Environment (.json)** | |
| **Berkas Environment tidak memuat kunci rahasia sungguhan** | [ ] Ya [ ] Tidak |
| **Repositori dapat dibuka dosen** | [ x ] Ya [ ] Tidak |

# Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

## Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

**Penjelasan:**

*Tulis jawaban Anda di sini.*

Menurut saya, assertion yang paling berguna adalah pengecekan status code karena dari situ saya bisa tahu apakah request berhasil atau mengalami kesalahan. Misalnya, saat menambahkan laporan bencana, status yang diharapkan adalah 201. Kalau statusnya berbeda, berarti ada bagian yang perlu saya periksa lagi.

## Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

**Penjelasan:**

*Tulis jawaban Anda di sini.*

Menurut saya, pengujian manual lebih mudah dilakukan untuk mengecek request satu per satu, sedangkan Collection Runner bisa menjalankan beberapa request sekaligus sehingga lebih menghemat waktu. Bagian yang paling sulit bagi saya adalah menulis test script karena harus menyesuaikan kode dengan respons dari API. Saya mengatasinya dengan melihat hasil respons di Postman, lalu memperbaiki script sampai hasil pengujiannya sesuai.

# Lampiran A  Daftar Periksa Sebelum Mengumpulkan

| No | Butir yang diperiksa | Ya |
|---|---|---|
| 1 | Seluruh tanda `[...]` sudah diganti dengan data API saya | Ya |
| 2 | Setiap endpoint API saya ada di Collection | Ya |
| 3 | Alamat memakai `{{base_url}}`, bukan ditulis langsung | Ya |
| 4 | Setiap request punya assertion status code dan struktur JSON | Ya |
| 5 | Nama setiap test berupa kalimat yang menyatakan harapan | Ya |
| 6 | POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya | Ya |
| 7 | Server dinyalakan ulang sebelum Runner dijalankan | Ya |
| 8 | Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas | Ya |
| 9 | Berkas Environment tidak memuat kunci rahasia sungguhan | Ya |
| 10 | Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS | Ya |

# Lampiran B  Kriteria Penilaian

*Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.*

| Kriteria | Deskripsi | Dinilai dari |
|---|---|---|
| Kelengkapan Postman Collection | Seluruh endpoint project akhir tercakup dalam satu Collection | Bab 1 dan Bab 2 |
| Ketepatan automated test script | Assertion status code dan struktur response JSON tepat sasaran | Bab 3 |
| Keberhasilan Collection Runner | Seluruh test lolos saat Collection dijalankan | Bab 4 |
| Kejelasan dokumentasi hasil | Tangkapan layar ringkasan pengujian disertakan dengan jelas | Bab 4 dan seluruh dokumen |